GUÍA PRÁCTICA FORMATIVA: PRUEBAS END-TO-END (E2E) CON PLAYWRIGHT

Ingeniería de Software V — Semana 10 (Período 2026-2)

Cúspide de la Pirámide: Auto-Waiting, Page Object Model (POM), Network Mocking y CI Headless

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS INTELIGENTES

DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS

INFORMACIÓN DE LA ACTIVIDAD — IDENTIFICACIÓN

CÓDIGO:	09776 - TIC	**PERÍODO:**	202620 (Semestre 2026-2)
NRC / GRUPO:	11005 / Grupo 001	**PONDERACIÓN:**	**Formativa** (Sin calificación sumativa) • Duración: 2 Horas
POLÍTICA IAG:	Nivel 3: Colaboración Asistida	**MODALIDAD:**	Parejas / Práctica de Laboratorio Guiada
ESTUDIANTE 1:	_____	**CÓDIGO:**	_____
ESTUDIANTE 2:	_____	**CÓDIGO:**	_____

1 Contexto del Problema & Aseguramiento de Calidad E2E

1.1 Descripción del Reto de Ingeniería de Software

En las sesiones anteriores consolidamos la base de la pirámide de calidad: pruebas unitarias con JUnit 5, cobertura con JaCoCo y análisis estático SAST con SonarQube dentro de pipelines de Jenkins. No obstante, una aplicación puede tener 100 % de cobertura unitaria y fallar ante el usuario real debido a incompatibilidades de navegador, renderizado CSS asíncrono o ruptura en los flujos de interacción del cliente web.

En esta guía práctica formativa, los estudiantes implementarán una suite profesional de pruebas **End-to-End (E2E)** con **Playwright** sobre la aplicación **Product Management CRUD** (React 18 + TypeScript + API REST). El objetivo es aprender a estructurar pruebas resilientes aplicando:

1. **Auto-waiting y locators de accesibilidad (W3C/ARIA):** Eliminar pausas fijas arbitrarias (`sleep`) usando selectores semánticos orientados al usuario.
2. **Page Object Model (POM):** Aislar la lógica de interacción con la UI de las aserciones de prueba para garantizar mantenibilidad.
3. **Network Mocking:** Interceptar llamadas HTTP (`page.route`) para validar la respuesta de la interfaz ante caídas del servidor sin alterar el backend.
4. **Validación opcional en CI/CD:** Ejecución desatendida (*headless*) en GitHub Actions con publicación de trazas forenses en caso de fallo.

Componente	Tecnología & Stack	Puerto / Endpoint
Aplicación Web (SPA)	React 18, Vite, TypeScript, TailwindCSS	Puerto 3000 (`http://localhost:3000`)
API REST Backend	Node Mock Server / Spring Boot REST	Puerto 8080 (`/api/v1/products`)
Suite E2E Playwright	Playwright Test Runner, TypeScript	Motores: Chromium, Firefox, WebKit
CI/CD (Opcional)	GitHub Actions (`e2e-playwright.yml`)	Modo Headless + HTML Report Artifacts

2 Procedimiento Experimental Paso a Paso

2.1 Fase 1: Puesta en Marcha del Código Base y Tooling (15 min)

Ubíquese en el directorio del código base suministrado (`codigo_base/`) e instale las dependencias de la aplicación y de Playwright:

```
cd codigo_base
# 1. Instalar dependencias del proyecto y Playwright
npm install

# 2. Descargar los navegadores oficiales de Playwright (Chromium, Firefox, WebKit)
npx playwright install --with-deps

# 3. Iniciar la aplicacion web y el servidor mock en segundo plano:
npm run dev
```

Verifique en su navegador local que la aplicación responde correctamente en `http://localhost:3000`.

2.2 Fase 2: Configuración Multi-Navegador en `playwright.config.ts` (15 min)

Inspeccione el archivo de configuración central `playwright.config.ts` asegurando los siguientes parámetros de ejecución:

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './e2e/tests',
  fullyParallel: true,
  retries: 1,                          // 1 reintento para aislar flakiness de red
  reporter: [['html', { open: 'never' }], ['list']],
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',           // Capturar traza forense al fallar
    screenshot: 'only-on-failure',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
  ],
});
```

2.3 Fase 3: Implementación del Page Object Model (POM) (30 min)

En `e2e/pages/ProductPage.ts`, construya la clase que encapsula los locators y métodos de interacción con el catálogo de productos. Utilice selectores de accesibilidad semántica:

```
import { Page, Locator } from '@playwright/test';

export class ProductPage {
  readonly page: Page;
  readonly nameInput: Locator;
  readonly priceInput: Locator;
  readonly submitButton: Locator;
  readonly successAlert: Locator;

  constructor(page: Page) {
    this.page = page;
    this.nameInput = page.getByLabel('Nombre del Producto');
    this.priceInput = page.getByLabel('Precio');
    this.submitButton = page.getByRole('button', { name: 'Registrar Producto' });
    this.successAlert = page.getByRole('alert');
  }
```

```
  async goto() {
    await this.page.goto('/');
  }

  async createProduct(name: string, price: string) {
    await this.nameInput.fill(name);
    await this.priceInput.fill(price);
    await this.submitButton.click();
  }
}
```

2.4 Fase 4: Pruebas de Interacción y Network Mocking (30 min)

En `e2e/tests/products.spec.ts`, escriba las pruebas para el flujo feliz y para la simulación de caída HTTP 500:

```
import { test, expect } from '@playwright/test';
import { ProductPage } from '../pages/ProductPage';

test.describe('Gestion de Productos - E2E Suite', () => {
  test('Flujo Feliz: Crear producto y verificar en la tabla', async ({ page }) => {
    const productPage = new ProductPage(page);
    await productPage.goto();

    await productPage.createProduct('Monitor 4K', '1250000');

    // Aserciones Web-First con auto-retry
    await expect(productPage.successAlert).toHaveText('Producto creado exitosamente');
    await expect(page.getByRole('cell', { name: 'Monitor 4K' })).toBeVisible();
  });

  test('Resiliencia: Manejar error 500 del backend mediante Network Mocking', async ({ page }) => {
    // Interceptar la llamada POST a la API REST
    await page.route('**/api/v1/products', async route => {
      if (route.request().method() === 'POST') {
        await route.fulfill({
          status: 500,
          contentType: 'application/json',
          body: JSON.stringify({ message: 'Error interno en la base de datos' })
        });
      } else {
        await route.continue();
      }
    });

    const productPage = new ProductPage(page);
    await productPage.goto();
    await productPage.createProduct('Teclado Mecanico', '350000');

    // Validar que el frontend muestra alerta de error y no se congela
    await expect(page.getByRole('alert')).toContainText('Error al procesar el producto');
  });
});
```

2.5 Fase 5: Diagnóstico con Trace Viewer y Ejecución Headless (15 min)

Ejecute la suite completa y examine las herramientas de diagnóstico:

```
# Ejecutar todas las pruebas en modo headless
npx playwright test

# En caso de fallo o inspeccion visual, abrir el reporte HTML interactivo:
npx playwright show-report
```

2.6 Fase 6 (OPCIONAL): Validación Automatizada en Pipeline CI/CD

Para los equipos que deseen integrar esta verificación en su pipeline de integración continua, cree el archivo `.github/workflows/e2e-playw`

```
name: E2E Playwright Suite
on: [push, pull_request]

jobs:
  test-e2e:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - name: Install dependencies & Playwright Browsers
        run: |
          npm ci
          npx playwright install --with-deps
      - name: Run E2E Tests Headless
        run: npm run test:e2e
      - name: Upload HTML Report on Failure
        uses: actions/upload-artifact@v4
        if: failure()
        with:
          name: playwright-report
          path: playwright-report/
```

3 Cuestionario de Reflexión Conceptual & Checklist Formativo

3.1 Preguntas de Reflexión Técnica

1. **Actionability y Auto-Waiting:** ¿De qué manera los mecanismos internos de Playwright (comprobación de visibilidad, estabilidad y eventos) eliminan la necesidad de usar `sleep()` arbitrarios en las pruebas?
2. **Locators W3C/ARIA vs XPath:** ¿Por qué escribir selectores basados en accesibilidad (`getByRole`, `getByLabel`) no solo hace las pruebas más resilientes a cambios de diseño, sino que promueve mejores prácticas de accesibilidad web?
3. **Network Mocking vs Pruebas de Integración Reales:** ¿En qué casos de prueba es conveniente interceptar peticiones de red y cuándo es obligatorio probar el backend real conectado a la base de datos?

3.2 Checklist de Logro Formativo (Autoevaluación)

Objetivo Formativo	Evidencia Observable	Estado
1. Configuración Multi-Motor	Playwright ejecuta pruebas limpiamente en Chromium, Firefox y WebKit.	[ ] Logrado
2. Page Object Model (POM)	Selectores y acciones encapsulados en clase `ProductPage.ts`.	[ ] Logrado
3. Network Mocking	Intercepción de fallo HTTP 500 y verificación de respuesta en la UI.	[ ] Logrado
4. Diagnóstico Forense	Inspección de reporte HTML y trazas paso a paso con Trace Viewer.	[ ] Logrado
5. CI/CD (Opcional)	Pipeline en GitHub Actions configurado con reporte en artefactos.	[ ] Opcional